

Verifying curve25519-dalek using Lean

Oliver Butterley, Markus Dablander, Alessandro D'Angelo, Hoang Le Truong, Liao Zhang



Rust Verification Workshop, Turin 14 April 2026

The Beneficial AI Foundation

Core strategy: making formal verification easier, cheaper and ubiquitous.

Lean-Dalek Team



Oliver Butterley

@oliver-butterley



Markus Dablander

@MarkusFerdinandDablander



Alessandro D'Angelo

@a-dangelo



Hoang Le Truong

@truonghoangle



Liao Zhang

@zhangliao714

Thanks to

Son Ho (Aeneas) and Guillaume Boisseau (Charon) and others for their ongoing support.

Overview

Real-world experience of verifying the functional correctness of production Rust cryptography code in Lean.

1. **Background:** verification target, toolchain, Lean proofs, summary
2. **Extraction challenges:** what breaks when you try to extract real Rust
3. **Proof engineering:** what breaks when you try to prove at scale
4. **Contributions** to open-source verification infrastructure

Project repo: <https://github.com/Beneficial-AI-Foundation/curve25519-dalek-lean-verify>

The task: curve25519-dalek

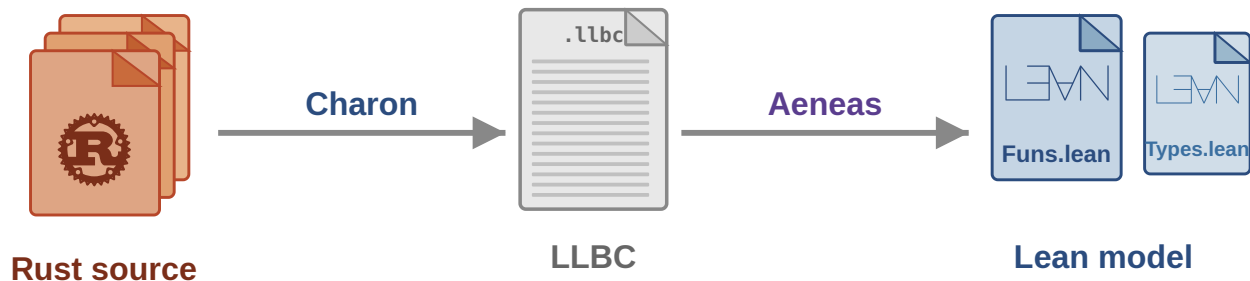
- Pure Rust library for elliptic curve primitives
- ~33k LOC
- We target serial/u64 backend
- E.g., Bit-level manipulations on $5 \times \text{u64}$ limbs
- E.g., constructors and operations that guarantee that curve points stay as curve points

```
/// Reduce to enforce bound  $2^{(51 + \epsilon)}$ .
fn reduce(mut limbs: [u64; 5]) → [u64; 5] {
    const LOW_51_BIT_MASK: u64 =
        (1u64 << 51) - 1;

    let c0 = limbs[0] >> 51;
    let c1 = limbs[1] >> 51;
    let c2 = limbs[2] >> 51;
    let c3 = limbs[3] >> 51;
    let c4 = limbs[4] >> 51;

    limbs[0] &= LOW_51_BIT_MASK;
    limbs[1] &= LOW_51_BIT_MASK;
    // ...
    limbs[0] += c4 * 19; //  $2^{255} \equiv 19$ 
    limbs[1] += c0;
    // ...
    limbs
}
```

Extraction: Rust to Lean



The Lean model faithfully captures Rust semantics:

- `Result` monad: operations can succeed (`ok`), fail (`fail`), or diverge (`div`)
- Arithmetic operations are fallible (overflow checking), etc.
- Imperative code via `do` -notation with monadic bind
- `Funs.lean` : 8,176 lines of extracted function definitions

Proofs in Lean

```
theorem reduce_spec (limbs : Array U64 5#usize) :  
  reduce limbs { (result : FieldElement51) =>  
    (∀ i < 5, result[i]!.val < 2 ^ 52) ∧  
    Field51_as_Nat limbs ≡ Field51_as_Nat result [MOD p] } := by  
  unfold reduce  
  step*  
  · simp [*]; scalar_tac          -- ⊢ ↑i15 + ↑i14 ≤ U64.max  
  · simp [*]; scalar_tac  
  · simp [*]; scalar_tac  
  · simp [*]; scalar_tac  
  · simp [*]; scalar_tac  
  · constructor  
  · intro i _                    -- ∀ i < 5, result limbs < 2^52  
    interval_cases i  
    all_goals simp [*]; scalar_tac  
  · simp [Field51_as_Nat, Finset.sum_range_succ, p, Nat.ModEq, *]  
    omega
```

The spec states: the function succeeds, all output limbs are bounded, and the result equals the input mod p . The proof unfolds the function, uses `step*` to process each extracted operation, then discharges overflow and correctness goals.

Lean specs

Full power and extensibility of Lean available for writing specs.

```
theorem add_spec
  (self other : EdwardsPoint) (hself : self.IsValid) (hother : other.IsValid) :
  add self other { (result : EdwardsPoint) ⇒
    result.IsValid ∧
    result.toPoint = self.toPoint + other.toPoint } := by ...
```

- Addition in extended twisted Edwards coordinates $P_i = (X_i, Y_i, Z_i, T_i)$.
- If inputs `IsValid` then the output `IsValid` (`InBounds` and `OnCurve`[†]).

Function computes $P_3 = P_1 + P_2$ where

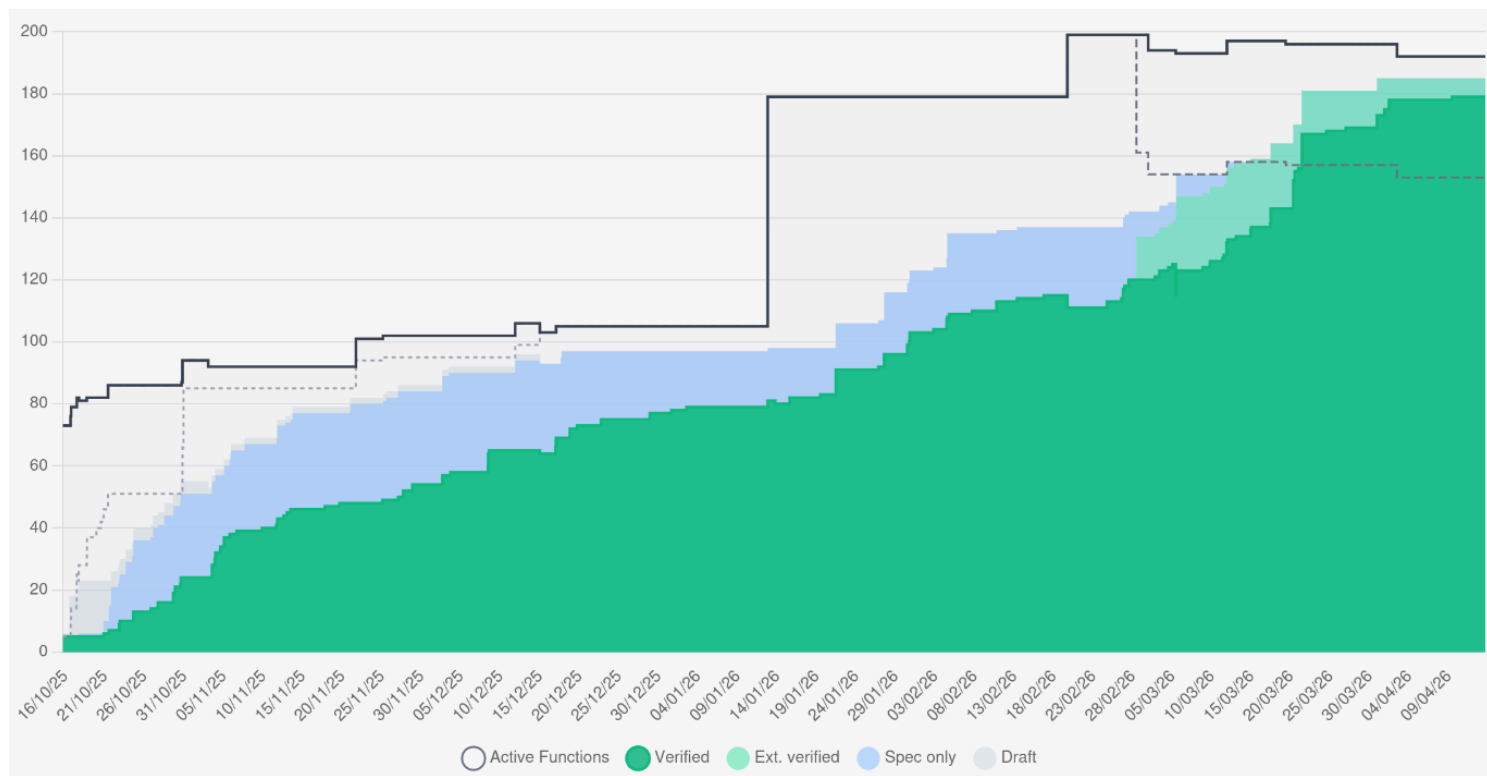
$$A = (Y_1 - X_1)(Y_2 - X_2), \quad B = (Y_1 + X_1)(Y_2 + X_2), \quad C = 2dT_1T_2, \quad D = 2Z_1Z_2$$

$$E = B - A, \quad F = D - C, \quad G = D + C, \quad H = B + A$$

$$X_3 = EF, \quad Y_3 = GH, \quad T_3 = EH, \quad Z_3 = FG$$

[†] $-X^2 + Y^2 = Z^2 + dT^2$ and $XY = ZT$.

Summary



Extraction challenges

- Initially some Rust wasn't supported by Charon/Aeneas (e.g., iterators)
- Some modifications to the source code (later reverted)
- Some tweaks to the output Lean code (later reverted)
- Some functions excluded from extraction
- CI for Aeneas from the beginning with tweaks applied after extraction

Proof engineering (1/2)

In general not a problem to complete proofs but the UX can be improved.

- Lean heartbeat escalation and slow proofs (it is possible to write terrible proofs!)
- Large number literals (e.g., 78-digit numbers for `ZMod` and kernel elaboration consumes 20+ GB RAM trying to normalise these)
- Long functions need to be broken into several pieces for convenience

Hide expensive hypotheses from `simp` to prevent timeouts:

```
private def Hold (P : Prop) : Prop := P -- opaque to simp

-- Before calling step on an expensive function:
have h_mod : Hold (... expensive ...) := h_original
clear h_original
step as {result, h_post}    -- simp doesn't touch Hold hypotheses
change ... expensive... at h_mod -- recover via definitional equality
```

Proof engineering (2/2)

Expensive unwrapping of array updates

```
▼ Tactic state
1 goal
├ (∀ i < 5, !limbs10[i]! < 2 ^ 52) ∧
  Field51_as_Nat limbs = Field51_as_Nat limbs10 [MOD p] ∧ Field51_as_Nat limbs10 < 2 * p
limbs10_post : limbs10 = limbs9.set 4#usize i25
_ : [> let limbs10 ← limbs9.update 4#usize i25 <]
limbs10 : Std.Array U64 5#usize
i25_post : ri25 = ri24 + rc3
_# : [> let i25 ← i24 + c3 <]
i25 : U64
i24_post : i24 = (!limbs9)[4]!
_# : [> let i24 ← limbs9.index_usize 4#usize <]
i24 : U64
limbs9_post : limbs9 = limbs8.set 3#usize i23
_# : [> let limbs9 ← limbs8.update 3#usize i23 <]
limbs9 : Std.Array U64 5#usize
i23_post : ri23 = ri22 + rc2
_# : [> let i23 ← i22 + c2 <]
i23 : U64
i22_post : i22 = (!limbs8)[3]!
_# : [> let i22 ← limbs8.index_usize 3#usize <]
i22 : U64
limbs8_post : limbs8 = limbs7.set 2#usize i21
_# : [> let limbs8 ← limbs7.update 2#usize i21 <]
limbs8 : Std.Array U64 5#usize
i21_post : ri21 = ri20 + rc1
_# : [> let i21 ← i20 + c1 <]
i21 : U64
i20_post : i20 = (!limbs7)[2]!
_# : [> let i20 ← limbs7.index_usize 2#usize <]
i20 : U64
limbs7_post : limbs7 = limbs6.set 1#usize i19
_# : [> let limbs7 ← limbs6.update 1#usize i19 <]
limbs7 : Std.Array U64 5#usize
i19_post : ri19 = ri18 + rc0
_# : [> let i19 ← i18 + c0 <]
i19 : U64
i18_post : i18 = (!limbs6)[1]!
_# : [> let i18 ← limbs6.index_usize 1#usize <]
i18 : U64
limbs6_post : limbs6 = limbs5.set 0#usize i17
_# : [> let limbs6 ← limbs5.update 0#usize i17 <]
limbs6 : Std.Array U64 5#usize
i17_post : ri17 = ri16 + ri15
_# : [> let i17 ← i16 + i15 <]
i17 : U64
i16_post : i16 = (!limbs5)[0]!
_# : [> let i16 ← limbs5.index_usize 0#usize <]
```

Trust model

To know that the specifications are satisfied, we must trust:

1. **The Lean proof checker:** Lean's minimal trusted kernel guarantees correctness
2. **The Aeneas translation:** extracted Lean code faithfully represents the Rust source
3. **The specifications:** written in Lean, readable by humans, drillable via interactive features
4. **External function specs:** dependencies modelled manually in `FunExternal.lean`

We use many things from Mathlib, this doesn't require additional trust.

Proximity of code and proof

- CI runs Aeneas and Lean: confirms extraction hasn't been tampered with and confirms proofs compile
- No specs or proofs in the Rust source code
- Dashboard to show the spec status

Upstreaming

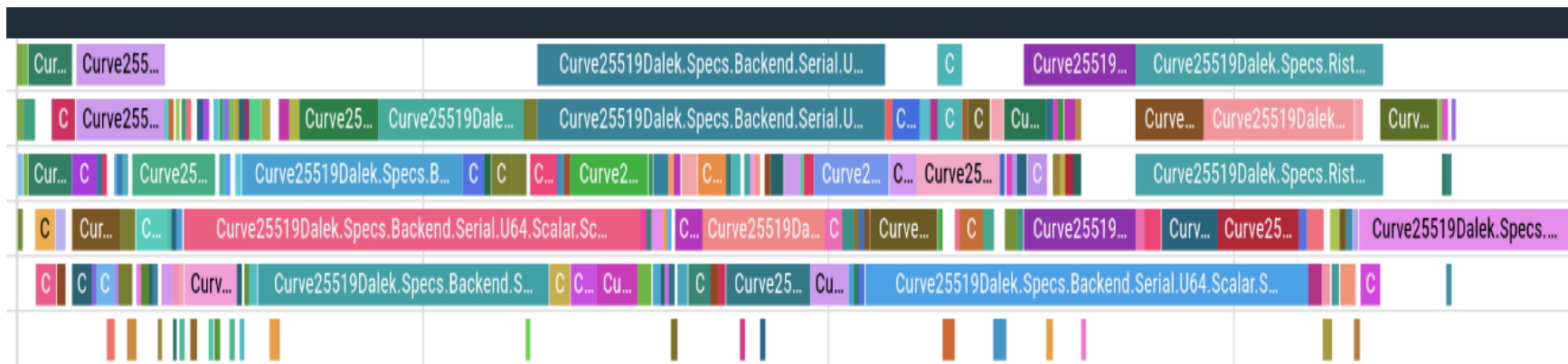
Project has contributed to Aeneas and Mathlib. More now that we are concluding.

Aim: Don't just prove a given spec theorem, instead figure out what can be added to the ecosystem so that similar proofs are easy.

- Improve Lean library (Aeneas, Mathlib)
- Improve proof automation (tactics)
- Improve skill files (or just more examples)

AI use

- Humans write the specs
- AI assists to write proofs
- Profilers and other tooling to assist
- Skill files and improved proof automation
- The proof checker guarantees correctness of the proof.



Leaderboard (of worst offenders)

Weekly Benchmark Report - 2026-04-13 #789

 Open



github-actions bot opened yesterday

Build Times

Total: 68m 8s

Module	Duration
Curve25519Dalek.Specs.Backend.Serial.U64.Scalar.Scalar52.ToBytes	5m 37s
Curve25519Dalek.Specs.Backend.Serial.U64.Scalar.Scalar52.MontgomeryReduce	5m 27s
Curve25519Dalek.Specs.Backend.Serial.U64.Field.FieldElement51.ToBytes	4m 17s
Curve25519Dalek.Specs.Backend.Serial.U64.Field.FieldElement51.Mul	3m 35s
Curve25519Dalek.Specs.Ristretto.RistrettoPoint.ElligatorRistrettoFlavor	3m 3s
Curve25519Dalek.Specs.Backend.Serial.U64.Field.FieldElement51.Pow2K	2m 43s
Curve25519Dalek.Specs.Backend.Serial.U64.Scalar.Scalar52.FromBytesWide	2m 36s
Curve25519Dalek.Specs.Ristretto.RistrettoPoint.Compress	1m 51s
Curve25519Dalek.Math.Montgomery.Representation	1m 47s
Curve25519Dalek.Specs.Backend.Serial.CurveModels.CompletedPoint.Add	1m 30s
Curve25519Dalek.Specs.Field.FieldElement51.SqrtRatio	1m 23s
Curve25519Dalek.Specs.Backend.Serial.U64.Field.FieldElement51.Reduce	1m 5s
Curve25519Dalek.Specs.Backend.Serial.U64.Scalar.Scalar52.Sub	1m 2s
Curve25519Dalek.Specs.Backend.Serial.CurveModels.NegativeNistPoint.Sub	1m 2s

Summary: toolchain works, lots of possible refinements to aid scalability

Thank you



Repo: <https://github.com/Beneficial-AI-Foundation/curve25519-dalek-lean-verify>